

LAB ASSIGNMENT – 13

NAME : SHREE PRIYA

2303A51465

BATCH – 29

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
 - Prompt AI to identify duplicate logic and replace it with functions or classes.
 - Ensure the refactored code maintains the same output.
 - Add docstrings to all functions.
- Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
 - Refactored code with a reusable function and no duplication.
 - Well documented code

PROMPT: Refactor the given Python script to remove duplicate rectangle calculations by creating reusable functions with proper docstrings, ensuring the output remains unchanged

```
#!/usr/bin/env python3
```

```
def calculate_area(length, width):
```

```
    """
```

```
    Calculate the area of a rectangle.
```

```
    Parameters:
```

```
    length (int or float): The length of the rectangle.
```

```
    width (int or float): The width of the rectangle.
```

Returns:

int or float: The calculated area of the rectangle.

"""

return length * width

def calculate_perimeter(length, width):

"""

Calculate the perimeter of a rectangle.

Parameters:

length (int or float): The length of the rectangle.

width (int or float): The width of the rectangle.

Returns:

int or float: The calculated perimeter of the rectangle.

"""

return 2 * (length + width)

def print_rectangle_details(length, width):

"""

Print the area and perimeter of a rectangle.

Parameters:

length (int or float): The length of the rectangle.

width (int or float): The width of the rectangle.

"""

print("Area of Rectangle:", calculate_area(length, width))

print("Perimeter of Rectangle:", calculate_perimeter(length, width))

Rectangle Data

rectangles = [

(5, 10),

(7, 12),

(10, 15)

]

Process each rectangle

for length, width in rectangles:

print_rectangle_details(length, width)

```
1 # Refactor the given Python script to remove duplicate rectangle calculations by creating reusable functions with proper docstrings, ensuring the output remains unchanged.
2 #!/usr/bin/env python
3 def calculate_area(length, width):
4     """
5     Calculate the area of a rectangle.
6     Parameters:
7     length (int or float): The length of the rectangle.
8     width (int or float): The width of the rectangle.
9
10    Returns:
11    int or float: The calculated area of the rectangle.
12    """
13    return length * width
14
15 def calculate_perimeter(length, width):
16     """
17     Calculate the perimeter of a rectangle.
18     Parameters:
19     length (int or float): The length of the rectangle.
20     width (int or float): The width of the rectangle.
21
22    Returns:
23    int or float: The calculated perimeter of the rectangle.
24    """
25    return 2 * (length + width)
26
27 def print_rectangle_details(length, width):
28     """
29     Print the area and perimeter of a rectangle.
30     Parameters:
31     length (int or float): The length of the rectangle.
32     width (int or float): The width of the rectangle.
33
34    """
35    print("Area of Rectangle:", calculate_area(length, width))
36    print("Perimeter of Rectangle:", calculate_perimeter(length, width))
37
38 # Rectangle Data
39 rectangles = [
40     (5, 10),
41     (7, 12),
42     (10, 15)
43 ]
44
45 # Process each rectangle
46 for length, width in rectangles:
47     print_rectangle_details(length, width)
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
 - Identify repeated or related logic and extract it into reusable functions.
 - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

Legacy script with inline repeated logic

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

- Expected Output:
 - Code with a function `calculate_total(price)` that can be reused for multiple price inputs. Well documented code

PROMPT : Refactor the legacy Python script by extracting the repeated tax calculation logic into a reusable function `calculate_total(price)` with proper docstrings while maintaining the same output

CODE:

```
def calculate_total(price):  
    """  
  
    Calculate the total price including 18% tax.  
  
    Parameters:  
  
    price (int or float): The original price of the product.  
  
    Returns:  
  
    float: Total price after adding 18% tax.  
    """  
  
    tax = price * 0.18  
    total = price + tax  
    return total  
  
# Prices to process  
prices = [250, 500]  
  
# Print total price for each item  
for price in prices:  
    print("Total Price:", calculate_total(price))
```

```
# Refactor the legacy Python script by extracting the repeated tax calculation logic into a reusable function calculate_total(price).
# [11] proper docstrings while maintaining the same output
def calculate_total(price):
    """
    Calculate the total price including 18% tax.

    Parameters:
    price (int or float): The original price of the product.

    Returns:
    float: Total price after adding 18% tax.
    """
    tax = price * 0.18
    total = price + tax
    return total

# Prices to process
prices = [250, 500]

# Print total price for each item
for price in prices:
    print("Total Price:", calculate_total(price))
```

DEVELOP OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> cd "C:\Users\Shree priya\Downloads"; & "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe"
2025.12.0-win32-x64\bundled\libs\debugpy\launcher" 88744: "C:\Users\Shree priya\Downloads\sac11.py"
Total Price: 295.0
Total Price: 590.0
PS C:\Users\Shree priya\Downloads>
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if-elif-else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - Accept marks as a parameter.
 - Return the corresponding grade as a string.
 - The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"
 - Marks $\geq 40 \rightarrow$ "Grade D"

- Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- The class
- The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.

Print the returned grade values.

PROMPT: Refactor the repeated grading conditional logic into an object-oriented Python solution using a class GradeCalculator

with a reusable method calculate_grade(self, marks) including proper docstrings

while preserving the exact grading conditions.

```

class GradeCalculator:
    """
    A class used to calculate student grades based on marks.

    Methods
    -----

    calculate_grade(self, marks):
        Returns the grade corresponding to the given marks.
    """

    def calculate_grade(self, marks):
        """
        Determine the grade based on the student's marks.

        Parameters:
        marks (int or float): The marks obtained by the student (0–100).

        Returns:
        str: The grade corresponding to the marks.
        """

        if 90 <= marks <= 100:
            return "Grade A"

        elif marks >= 80:
            return "Grade B"

        elif marks >= 70:
            return "Grade C"

        elif marks >= 40:
            return "Grade D"

        elif marks >= 0:
            return "Fail"

# Create object of GradeCalculator
grade_calculator = GradeCalculator()

# Call method for different students
marks_list = [85, 72, 95, 38]

```

for marks in marks_list:

```
print(f"Marks: {marks} → {grade_calculator.calculate_grade(marks)}")
```

```
1 # Refactor the repeated grading conditional logic into an object-oriented Python solution using a class GradeCalculator
2 # with a reusable method calculate_grade(self, marks) including proper docstrings
3 # while preserving the exact grading conditions.
4 class GradeCalculator:
5     """
6     A class used to calculate student grades based on marks.
7
8     Methods
9     """
10    calculate_grade(self, marks):
11        """
12        Returns the grade corresponding to the given marks.
13        """
14    def calculate_grade(self, marks):
15        """
16        Determine the grade based on the student's marks.
17
18        Parameters:
19        marks (int or float): The marks obtained by the student (0-100).
20
21        Returns:
22        str: The grade corresponding to the marks.
23        """
24        if 90 <= marks <= 100:
25            return "Grade A"
26        elif marks >= 80:
27            return "Grade B"
28        elif marks >= 70:
29            return "Grade C"
30        elif marks >= 60:
31            return "Grade D"
32        elif marks >= 0:
33            return "Fail"
34
35    # Create object of GradeCalculator
36    grade_calculator = GradeCalculator()
37
38    # Call method for different students
39    marks_list = [85, 72, 95, 38]
40
41    for marks in marks_list:
42        print(f"Marks: {marks} → {grade_calculator.calculate_grade(marks)}")
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
PS C:\Users\Shree priya\Downloads> c::; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\
-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '60744' '--' 'C:\Users\Shree priya\Downloads\aac13.py'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\Shree priya\Downloads>
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```


- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

PROMPT: Refactor the procedural Python script by separating input, processing, and output logic into modular functions `get_input()`, `calculate_square()`, and `display_result()` without changing the program behavior

CODE:

```
def get_input():
```

```
    """
```

```
    Get an integer input from the user.
```

```
    Returns:
```

```
    int: The number entered by the user.
```

```
    """
```

```
    return int(input("Enter number: "))
```

```
def calculate_square(number):
```

```
    """
```

```
    Calculate the square of a given number.
```

```
    Parameters:
```

```
    number (int): The number to be squared.
```

```
    Returns:
```

```
    int: The square of the number.
```

```
    """
```

```
    return number * number
```

```
def display_result(result):
```

```
    """
```

```
    Display the square result.
```

```
    Parameters:
```

```
    result (int): The calculated square value.
```

```
    """
```

```
    print("Square:", result)
```

```
# Main Execution Flow
```

```
num = get_input()
```

```
square = calculate_square(num)
```

```
display_result(square)
```

```
1 # Refactor the procedural Python script by separating input, processing, and output logic into modular
2 # functions get_input(), calculate_square(), and display_result() without changing the program behavior
3 def get_input():
4     """
5     Get an integer input from the user.
6
7     Returns:
8     int: The number entered by the user.
9     """
10    return int(input("Enter number: "))
11
12
13 def calculate_square(number):
14     """
15     Calculate the square of a given number.
16
17     Parameters:
18     number (int): The number to be squared.
19
20     Returns:
21     int: The square of the number.
22     """
23    return number * number
24
25
26 def display_result(result):
27     """
28     Display the square result.
29
30     Parameters:
31     result (int): The calculated square value.
32     """
33    print("Square:", result)
34
35 # Main Execution Flow
36 num = get_input()
37 square = calculate_square(num)
38 display_result(square)
39
```

PROBLEMS OUTPUT TERMINAL PORTS

✓ TERMINAL

```
PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\
-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51673' '--' 'C:\Users\Shree priya\Downloads\aac13.py'
Enter number: 8
Square: 64
PS C:\Users\Shree priya\Downloads> 
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes

PROMPT: # Refactor the procedural salary and tax calculation script into an object-oriented design

using a class EmployeeSalaryCalculator that applies encapsulation with attributes and methods while preserving the original functionality

CODE:

```
class EmployeeSalaryCalculator:
```

```
    """
```

A class to calculate employee net salary after tax deduction.

Attributes:

salary (float): The gross salary of the employee.

tax_rate (float): The tax rate applied to the salary.

```
    """
```

```
    def __init__(self, salary, tax_rate=0.2):
```

```
        """
```

Initialize the EmployeeSalaryCalculator with salary and tax rate.

Parameters:

salary (float): The gross salary of the employee.

tax_rate (float, optional): The tax rate (default is 20%).

```
        """
```

```
        self.salary = salary
```

```
        self.tax_rate = tax_rate
```

```
    def calculate_tax(self):
```

```
        """
```

Calculate the tax amount.

Returns:

float: Calculated tax amount.

```
        """
```

```
        return self.salary * self.tax_rate
```

```
def calculate_net_salary(self):
```

```
    """
```

```
    Calculate the net salary after deducting tax.
```

```
    Returns:
```

```
    float: Net salary after tax deduction.
```

```
    """
```

```
    tax = self.calculate_tax()
```

```
    return self.salary - tax
```

```
# Create object
```

```
employee = EmployeeSalaryCalculator(50000)
```

```
# Print net salary
```

```
print(employee.calculate_net_salary())
```

```
1 # Refactor the procedural salary and tax calculation script into an object-oriented design
2 # using a class EmployeeSalaryCalculator that applies e
3 # ncapsulation with attributes and methods while preserving the original functionality
4 class EmployeeSalaryCalculator:
5     """
6     A class to calculate employee net salary after tax deduction.
7
8     Attributes:
9     salary (float): The gross salary of the employee.
10    tax_rate (float): The tax rate applied to the salary.
11    """
12    def __init__(self, salary, tax_rate=0.2):
13        """
14        Initialize the EmployeeSalaryCalculator with salary and tax rate.
15        Parameters:
16        salary (float): The gross salary of the employee.
17        tax_rate (float, optional): The tax rate (default is 20%).
18        """
19        self.salary = salary
20        self.tax_rate = tax_rate
21    def calculate_tax(self):
22        """
23        Calculate the tax amount.
24
25        Returns:
26        float: Calculated tax amount.
27        """
28        return self.salary * self.tax_rate
29    def calculate_net_salary(self):
30        """
31        Calculate the net salary after deducting tax.
32
33        Returns:
34        float: Net salary after tax deduction.
35        """
36        tax = self.calculate_tax()
37        return self.salary - tax
38
39 # Create object
40 employee = EmployeeSalaryCalculator(50000)
41 # Print net salary
42 print(employee.calculate_net_salary())
```

ROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:: cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\SelfUpdates\2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52427' '--' 'C:\Users\Shree priya\Downloads\aac13.py'
40000.0
PS C:\Users\Shree priya\Downloads> 
```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
if u == name:
    found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification

PROMPT: # Refactor the inefficient linear username search to use an optimized data structure like a set or dictionary to achieve $O(1)$ average time complexity with proper justification

CODE:

```
def is_username_taken(username, existing_usernames):
```

```
    """
```

```
    Checks if the given username is already taken.
```

```
    """
```

```
    # Convert the list of existing usernames to a set for  $O(1)$  average time complexity
```

```
    username_set = set(existing_usernames)
```

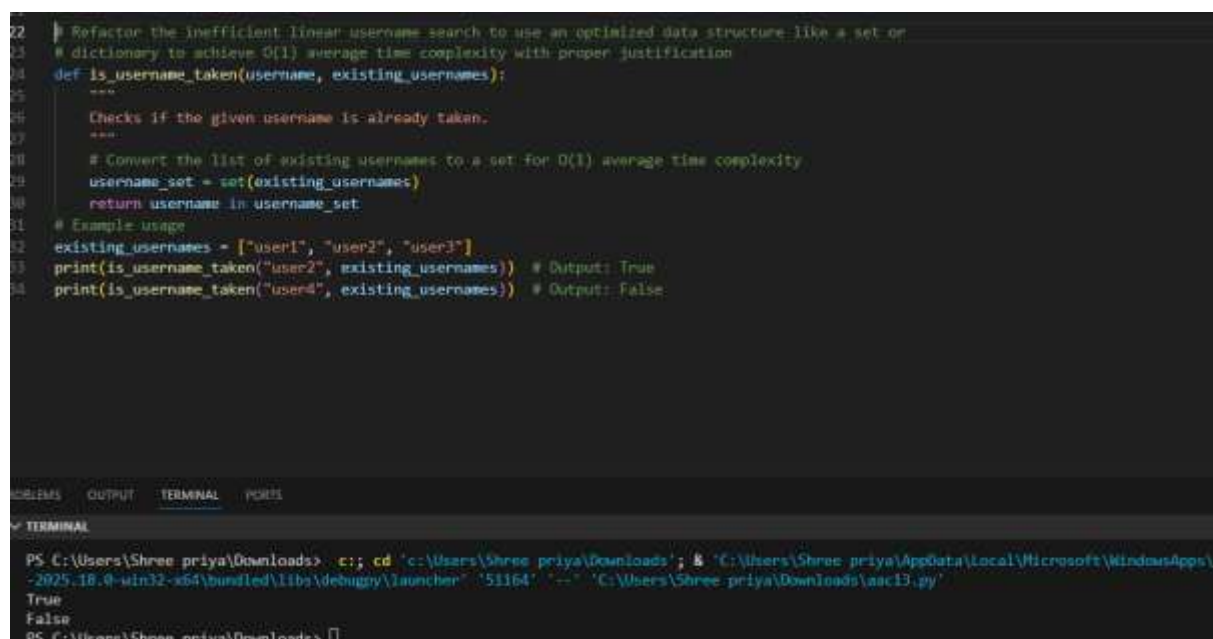
```
    return username in username_set
```

```
# Example usage
```

```
existing_usernames = ["user1", "user2", "user3"]
```

```
print(is_username_taken("user2", existing_usernames)) # Output: True
```

```
print(is_username_taken("user4", existing_usernames)) # Output: False
```



```
22 # Refactor the inefficient linear username search to use an optimized data structure like a set or
23 # dictionary to achieve  $O(1)$  average time complexity with proper justification
24 def is_username_taken(username, existing_usernames):
25     """
26     Checks if the given username is already taken.
27     """
28     # Convert the list of existing usernames to a set for  $O(1)$  average time complexity
29     username_set = set(existing_usernames)
30     return username in username_set
31 # Example usage
32 existing_usernames = ["user1", "user2", "user3"]
33 print(is_username_taken("user2", existing_usernames)) # Output: True
34 print(is_username_taken("user4", existing_usernames)) # Output: False
```

TERMINAL

```
PS C:\Users\Shree priya\Downloads> .\aaci3.py
True
False
PS C:\Users\Shree priya\Downloads>
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
 - add_book(title, author, isbn)
 - remove_book(isbn)
 - search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}

# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
```

```

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

```

CODE:

"""

Library Management Module

Provides functions to add, remove, and search books
using ISBN as a unique identifier.


```
"""
```

```
library_db = {}
```

```
def add_book(title, author, isbn):
```

```
    """
```

```
    Add a new book to the library database.
```

```
    Parameters:
```

```
    title (str): Title of the book.
```

```
    author (str): Author of the book.
```

```
    isbn (str): Unique ISBN number of the book.
```

```
    Returns:
```

```
    str: Status message indicating success or duplication.
```

```
    """
```

```
    if isbn not in library_db:
```

```
        library_db[isbn] = {"title": title, "author": author}
```

```
        return "Book added successfully."
```

```
    return "Book already exists."
```

```
def remove_book(isbn):
```

```
    """
```

```
    Remove a book from the library database using ISBN.
```

```
    Parameters:
```

```
    isbn (str): ISBN number of the book to remove.
```

```
    Returns:
```

```
    str: Status message indicating success or failure.
```

```
    """
```

```
    if isbn in library_db:
```

```
        del library_db[isbn]
```

```
    return "Book removed successfully."

    return "Book not found."
```

```
def search_book(isbn):
```

```
    """
```

Search for a book in the library database using ISBN.

Parameters:

isbn (str): ISBN number of the book to search.

Returns:

dict or str: Book details if found, otherwise a not found message.

```
    """
```

```
    if isbn in library_db:
```

```
        return library_db[isbn]
```

```
    return "Book not found."
```

main.py:

```
import library
```

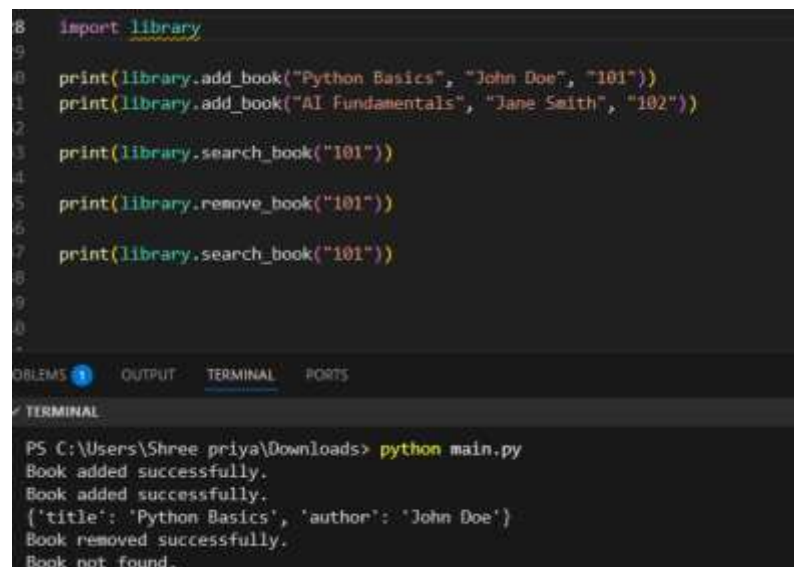
```
print(library.add_book("Python Basics", "John Doe", "101"))
```

```
print(library.add_book("AI Fundamentals", "Jane Smith", "102"))
```

```
print(library.search_book("101"))
```

```
print(library.remove_book("101"))
```

```
print(library.search_book("101"))
```



The screenshot shows a code editor with a dark theme. The top part displays the code for main.py, which imports the library module and performs several operations: adding two books, searching for a book by ISBN, removing a book, and searching again. The bottom part shows the terminal output, which matches the code's execution: two successful book additions, a search result for 'Python Basics', a successful removal, and a 'Book not found' message.

```
8  import library
9
10 print(library.add_book("Python Basics", "John Doe", "101"))
11 print(library.add_book("AI Fundamentals", "Jane Smith", "102"))
12
13 print(library.search_book("101"))
14
15 print(library.remove_book("101"))
16
17 print(library.search_book("101"))
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

✓ TERMINAL

```
PS C:\Users\Shree priya\Downloads> python main.py
Book added successfully.
Book added successfully.
{'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
```

```
1 """
2 Library Management Module
3 Provides functions to add, remove, and search books
4 using ISBN as a unique identifier.
5 """
6 library_db = {}
7 def add_book(title, author, isbn):
8     """
9     Add a new book to the library database.
10
11     Parameters:
12     title (str): Title of the book.
13     author (str): Author of the book.
14     isbn (str): Unique ISBN number of the book.
15
16     Returns:
17     str: Status message indicating success or duplication.
18     """
19     if isbn not in library_db:
20         library_db[isbn] = {"title": title, "author": author}
21         return "Book added successfully."
22     return "Book already exists."
23 def remove_book(isbn):
24     """
25     Remove a book from the library database using ISBN.
26
27     Parameters:
28     isbn (str): ISBN number of the book to remove.
29
30     Returns:
31     str: Status message indicating success or failure.
32     """
33     if isbn in library_db:
34         del library_db[isbn]
35         return "Book removed successfully."
36     return "Book not found."
37 def search_book(isbn):
38     """
39     Search for a book in the library database using ISBN.
40
41     Parameters:
42     isbn (str): ISBN number of the book to search.
43 """
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> python -m pydoc -w library
wrote library.html
PS C:\Users\Shree priya\Downloads>
```

Indexing completed

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
```

```
n=int(input("Enter limit: "))
```

```
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```

PROMPT: # Refactor the procedural Fibonacci code into a modular reusable function generate_fibonacci(n) with proper docstrings and test cases while improving structure and efficiency.

CODE:

```
def generate_fibonacci(n):
```

```
    """
```

```
    Generate Fibonacci series up to n terms.
```

```
    Parameters:
```

```
    n (int): Number of terms to generate.
```

```
    Returns:
```

```
    list: A list containing the Fibonacci sequence up to n terms.
```

```
    """
```

```
    if n <= 0:
```

```
        return []
```

```
    if n == 1:
```

```
        return [0]
```

```
    sequence = [0, 1]
```

```
    for _ in range(2, n):
```

```
        next_value = sequence[-1] + sequence[-2]
```

```
        sequence.append(next_value)
```

return sequence

Test Cases

print("Test Case 1 (n=1):", generate_fibonacci(1))

print("Test Case 2 (n=5):", generate_fibonacci(5))

print("Test Case 3 (n=10):", generate_fibonacci(10))

```
1 # Refactor the procedural fibonacci code into a modular reusable function generate_fibonacci(n) with
2 # proper docstrings and test cases while improving structure and efficiency.
3 def generate_fibonacci(n):
4     """
5     Generate Fibonacci series up to n terms.
6
7     Parameters:
8     n (int): Number of terms to generate.
9
10    Returns:
11    list: A list containing the Fibonacci sequence up to n terms.
12    """
13    if n <= 0:
14        return []
15
16    if n == 1:
17        return [0]
18
19    sequence = [0, 1]
20
21    for _ in range(2, n):
22        next_value = sequence[-1] + sequence[-2]
23        sequence.append(next_value)
24
25    return sequence
26
27
28 # Test Cases
29 print("Test Case 1 (n=1):", generate_fibonacci(1))
30 print("Test Case 2 (n=5):", generate_fibonacci(5))
31 print("Test Case 3 (n=10):", generate_fibonacci(10))
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> c:: cd "C:\Users\Shree priya\Downloads"; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\Common-IntelliSense\Python\Python\python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60354' '--' 'C:\Users\Shree priya\Downloads'
Test Case 1 (n=1): [0]
Test Case 2 (n=5): [0, 1, 1, 2, 3]
Test Case 3 (n=10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\Shree priya\Downloads>
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.

- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

PROMPT: # Refactor the procedural twin prime checker into optimized reusable functions `is_prime(n)` and `is_twin_prime(p1, p2)` with proper docstrings and generate all twin primes within a given range.

CODE:

```
import math

def is_prime(n):
    """
    Check whether a number is prime.

    Parameters:
    n (int): The number to check.

    Returns:
    bool: True if n is prime, otherwise False.
    """
    if n <= 1:
        return False
    if n <= 3:
```

```

        return True
    if n % 2 == 0:
        return False

    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1, p2):
    """
    Check whether two numbers form a twin prime pair.
    Parameters:
    p1 (int): First number.
    p2 (int): Second number.

    Returns:
    bool: True if both are prime and differ by 2.
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def generate_twin_primes(start, end):
    """
    Generate all twin prime pairs within a given range.
    Parameters:
    start (int): Starting value of range.
    end (int): Ending value of range.
    Returns:
    list: List of twin prime pairs.
    """
    twin_primes = []
    for num in range(start, end - 1):

```

```

    if is_twin_prime(num, num + 2):
        twin_primes.append((num, num + 2))

return twin_primes

# Example Usage

print("Twin primes between 1 and 50:")

print(generate_twin_primes(1, 50))

```

```

1 # Refactor the procedural twin prime checker into optimized reusable functions is_prime(n) and
2 # is_twin_prime(p1, p2) with proper docstrings and generate all twin primes within a given range.
3 import math
4 def is_prime(n):
5     """
6     Check whether a number is prime.
7     Parameters:
8     n (int): The number to check.
9     Returns:
10    bool: True if n is prime, otherwise False.
11    """
12    if n <= 1:
13        return False
14    if n <= 3:
15        return True
16    if n % 2 == 0:
17        return False
18    for i in range(3, int(math.sqrt(n)) + 1, 2):
19        if n % i == 0:
20            return False
21    return True
22 def is_twin_prime(p1, p2):
23     """
24     Check whether two numbers form a twin prime pair.
25     Parameters:
26     p1 (int): First number.
27     p2 (int): Second number.
28     Returns:
29     bool: True if both are prime and differ by 2.
30     """
31    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
32 def generate_twin_primes(start, end):
33     """
34     Generate all twin prime pairs within a given range.
35     Parameters:
36     start (int): Starting value of range.
37     end (int): Ending value of range.
38     Returns:
39     list: List of twin prime pairs.
40     """
41    twin_primes = []
42    for num in range(start, end + 1):
43        if is_twin_prime(num, num + 2):
44            twin_primes.append((num, num + 2))
45    return twin_primes
46 # Example Usage
47 print("Twin primes between 1 and 50:")
48 print(generate_twin_primes(1, 50))

```

```

PS C:\Users\Shree priya\Downloads> cd 'C:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'C:\Users\Shree priya\Downloads\launcher' '01000' -c 'C:\Users\Shree priya\Downloads\test11.py'
Twin primes between 1 and 50:
[(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43)]
PS C:\Users\Shree priya\Downloads>

```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`

2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

PROMPT: Refactor the procedural Chinese Zodiac program into a modular implementation using a reusable function `get_zodiac(year)`, replace the if-elif chain with a cleaner data structure, add docstrings, and separate input handling from logic.

CODE:

```
def get_zodiac(year):
```

```
    """
```

```
    Determine the Chinese Zodiac sign for a given year.
```

```
    Parameters:
```

```
    year (int): The year for which the zodiac sign is required.
```

```
    Returns:
```

```
    str: The corresponding Chinese Zodiac sign.
```

```
    """
```

```
    zodiac_cycle = [
```

```
        "Monkey", "Rooster", "Dog", "Pig",
```

```
        "Rat", "Ox", "Tiger", "Rabbit",
```

```
        "Dragon", "Snake", "Horse", "Goat"
```

```
    ]
```

```
    return zodiac_cycle[year % 12]
```

```
def main():
```

```
    """
```

```
    Handle user input and display the zodiac sign.
```

```
    """
```

```
    year = int(input("Enter a year: "))
```

```
    sign = get_zodiac(year)
```

```
print(sign)
```

Program Execution

```
if __name__ == "__main__":
```

```
    main()
```

```
1 # Refactor the procedural Chinese zodiac program into a modular implementation using a reusable function
2 # get_zodiac(year), replace the if-elif chain with a cleaner data structure, add docstrings, and separate input handling from logic.
3 def get_zodiac(year):
4     """
5     Determine the Chinese Zodiac sign for a given year.
6
7     Parameters:
8     year (int): The year for which the zodiac sign is required.
9
10    Returns:
11    str: The corresponding Chinese Zodiac sign.
12    """
13    zodiac_cycle = [
14        "Monkey", "Rooster", "Dog", "Pig",
15        "Rat", "Ox", "Tiger", "Rabbit",
16        "Dragon", "Snake", "Horse", "Goat"
17    ]
18
19    return zodiac_cycle[year % 12]
20
21 def main():
22     """
23     Handle user input and display the zodiac sign.
24     """
25
26     year = int(input("Enter a year: "))
27     sign = get_zodiac(year)
28     print(sign)
29
30 # Program Execution
31 if __name__ == "__main__":
32     main()
```

OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> cd 'C:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62508' '-.' 'C:\Users\Shree priya\Downloads\aac11.py'
Enter a year: 2003
Goat
PS C:\Users\Shree priya\Downloads>
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ ✓ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ ✗ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values

- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - Accept an integer parameter.
 - Return True if the number is divisible by the sum of its digits.
 - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

PROMPT: Refactor the procedural Harshad number checker into a clean reusable function `is_harshad(number)` with proper docstrings, separation of input and logic, boolean return values, and edge-case handling

CODE:

```
def is_harshad(number):
```

```
    """
```

Determine whether a given number is a Harshad (Niven) number.

A Harshad number is divisible by the sum of its digits.

Parameters:

number (int): The number to check.

Returns:

bool: True if the number is a Harshad number, otherwise False.

```
"""
# Handle negative numbers
number = abs(number)

# Edge case: 0 is not considered a Harshad number
if number == 0:
    return False

# Calculate sum of digits
digit_sum = sum(int(digit) for digit in str(number))

# Avoid division by zero
if digit_sum == 0:
    return False

return number % digit_sum == 0

def main():
    """
    Handle user input and display whether the number is Harshad or not.
    """
    num = int(input("Enter a number: "))
    print(is_harshad(num))

if __name__ == "__main__":
    main()
```

```
1 # Refactor the procedural Harshad number checker into a clean reusable function is_harshad(number) with proper docstrings,
2 # separation of input and logic, boolean return values, and edge-case handling
3 def is_harshad(number):
4     """
5     Determine whether a given number is a Harshad (Niven) number.
6
7     A Harshad number is divisible by the sum of its digits.
8
9     Parameters:
10     number (int): The number to check.
11
12     Returns:
13     bool: True if the number is a Harshad number, otherwise False.
14     """
15     # Handle negative numbers
16     number = abs(number)
17
18     # Edge case: 0 is not considered a Harshad number
19     if number == 0:
20         return False
21     # Calculate sum of digits
22     digit_sum = sum(int(digit) for digit in str(number))
23
24     # Avoid division by zero
25     if digit_sum == 0:
26         return False
27     return number % digit_sum == 0
28
29 def main():
30     """
31     Handle user input and display whether the number is Harshad or not.
32     """
33     num = int(input("Enter a number: "))
34     print(is_harshad(num))
35
36 if __name__ == "__main__":
37     main()
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
PS C:\Users\Shree priya\Downloads> ^C
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Wind
ython.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '54083' ^-- 'C:\Users\Shree priya\Downloads\aac13.py'
Enter a number: 8
True
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - Accept a non-negative integer `n`.
 - Return the number of trailing zeros in **`n!`**.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

PROMPT: Refactor the factorial trailing zeros program into an optimized reusable function `count_trailing_zeros(n)` using the mathematical approach of counting multiples of 5 without computing the full factorial.

CODE:

```
def count_trailing_zeros(n):
```

```
    """
```

```
    Calculate the number of trailing zeros in n! (factorial of n)
    without computing the full factorial.
```

```
    The number of trailing zeros is determined by counting
    the number of factors of 5 in n!.
```

```
    Parameters:
```

```
    n (int): A non-negative integer.
```

```
    Returns:
```

```
    int: Number of trailing zeros in n!.
```

```
        Returns 0 for n = 0.
```

```
        Returns None for negative input.
```

```
    """
```

```
    if n < 0:
```

```
        return None # Invalid input
count = 0
divisor = 5

while n // divisor > 0:
    count += n // divisor
    divisor *= 5

return count
def main():
    """
    Handle user input and display trailing zeros result.
    """
    n = int(input("Enter a number: "))
    result = count_trailing_zeros(n)
    if result is None:
        print("Invalid input. Please enter a non-negative number.")
    else:
        print("Trailing zeros:", result)
if __name__ == "__main__":
    main()
```



```
1 # Refactor the factorial trailing zeros program into an optimized reusable function count_trailing_zeros(n)
2 # using the mathematical approach of counting multiples of 5 without computing the full factorial.
3 def count_trailing_zeros(n):
4     """
5     Calculate the number of trailing zeros in n! (factorial of n)
6     without computing the full factorial.
7
8     The number of trailing zeros is determined by counting
9     the number of factors of 5 in n!.
10
11     Parameters:
12     n (int): A non-negative integer.
13
14     Returns:
15     int: Number of trailing zeros in n!.
16         Returns 0 for n = 0.
17         Returns None for negative input.
18     """
19     if n < 0:
20         return None # Invalid input.
21     count = 0
22     divisor = 5
23
24     while n // divisor > 0:
25         count += n // divisor
26         divisor *= 5
27
28     return count
29
30 def main():
31     """
32     Handle user input and display trailing zeros result.
33     """
34     n = int(input("Enter a number: "))
35     result = count_trailing_zeros(n)
36     if result is None:
37         print("Invalid input. Please enter a non-negative number.")
38     else:
39         print("Trailing zeros:", result)
40
41 if __name__ == "__main__":
42     main()
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:\cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\python\debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61557' -- "C:\Users\Shree priya\Downloads\aac13.py"
Enter a number: 6
Trailing zeros: 1
PS C:\Users\Shree priya\Downloads>
```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
 - Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
 - Edge: $1 \rightarrow [1]$
 - Negative: -5
 - Large: 27 (well-known long sequence)
 - Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow$ next = $n / 2$.
 - If n is odd $\rightarrow$ next = $3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

CODE:

```
def generate_collatz(n):
    if n <= 0:
        raise ValueError("Input must be positive")

    sequence = [n]

    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        sequence.append(n)

    return sequence

import pytest
from collatz import generate_collatz

def test_normal():
    assert generate_collatz(6) == [6,3,10,5,16,8,4,2,1]

def test_edge():
    assert generate_collatz(1) == [1]

def test_negative():
    with pytest.raises(ValueError):
        generate_collatz(-5)

def test_large():
    result = generate_collatz(27)
```

```
assert result[0] == 27
assert result[-1] == 1
```

```
[notice] To update, run: C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_
h25n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\Shree priya\OneDrive\Desktop\CollatzProject> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Shree priya\OneDrive\Desktop\CollatzProject
collected 0 items

===== no tests ran in 0.02s =====
PS C:\Users\Shree priya\OneDrive\Desktop\CollatzProject> dir

Directory: C:\Users\Shree priya\OneDrive\Desktop\CollatzProject

Mode                LastWriteTime         Length Name
----                -
dar--l             27-02-2026  10:36 AM             .pytest_cache
-a---l             27-02-2026  10:33 AM             278 collatz.py.txt
-a---l             27-02-2026  10:34 AM             394 test_collatz.py.txt

PS C:\Users\Shree priya\OneDrive\Desktop\CollatzProject> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Shree priya\OneDrive\Desktop\CollatzProject
collected 0 items

===== no tests ran in 0.01s =====
PS C:\Users\Shree priya\OneDrive\Desktop\CollatzProject> |
```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)
- Test Cases to Design:
 - Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
 - Edge: $1 \rightarrow [2]$
 - Negative: $-5 \rightarrow \text{Error}$
 - Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

PROMPT: Generate Lucas sequence up to n terms and validate using pytest including normal, edge, negative, and large test cases.

```
def generate_lucas(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return []
    if n == 1:
        return [2]

    lucas = [2, 1]
    for _ in range(2, n):
        lucas.append(lucas[-1] + lucas[-2])
    return lucas
```

```
n = int(input("Enter number of terms: "))
print(generate_lucas(n))
```

```

2 # Prompt: Generate lucas sequence up to n terms and validate using pytest including normal, edge, negative, and large test cases.
3
4 def generate_lucas(n):
5     if n < 0:
6         raise ValueError("n must be non-negative")
7     if n == 0:
8         return []
9     if n == 1:
10        return [2]
11
12    lucas = [2, 1]
13    for _ in range(2, n):
14        lucas.append(lucas[-1] + lucas[-2])
15    return lucas
16
17 n = int(input("Enter number of terms: "))
18 print(generate_lucas(n))

```

Terminal output:

```

PS C:\Users\Shree priya\Downloads> ^C
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> cd "c:\Users\Shree priya\Downloads" & "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe"
python.debugpy-2025.10.0-win32-a64\bundled\lib\debugpy\launcher" -51358" -- "C:\Users\Shree priya\Downloads\anc13.py"
Enter number of terms: 12
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 121, 199]
PS C:\Users\Shree priya\Downloads>

```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
 - Normal: "hello" → (2,3)
 - Edge: "" → (0,0)
 - Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

PROMPT: Count vowels and consonants and print output

CODE:

```
def count_vowels_consonants(text):
    vowels = "aeiouAEIOU"
    v = c = 0
```

```

    for ch in text:
        if ch.isalpha():
            if ch in vowels:
                v += 1
            else:
                c += 1
    return v, c

```

```

text = input("Enter text: ")
vowels, consonants = count_vowels_consonants(text)
print("Vowels:", vowels)
print("Consonants:", consonants)

```

```
21 # Count vowels and consonants and print output
22 def count_vowels_consonants(text):
23     vowels = "aeiouAEIOU"
24     v = c = 0
25
26     for ch in text:
27         if ch.isalpha():
28             if ch in vowels:
29                 v += 1
30             else:
31                 c += 1
32     return v, c
33
34
35 text = input("Enter text: ")
36 vowels, consonants = count_vowels_consonants(text)
37 print("Vowels:", vowels)
38 print("Consonants:", consonants)
39
```

PROBLEMS 2 OUTPUT TERMINAL PORTS

▼ TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c;; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56502' '--' 'C:\Users\Shree priya\D
Enter text: AAYA SHER
Vowels: 4
Consonants: 4
PS C:\Users\Shree priya\Downloads> 
```

JUSTIFICATIONS:

Task 1 – Removing Code Duplication

Refactoring removes repeated logic and improves maintainability.

Reusable functions reduce redundancy and make the code scalable.

Well-documented functions improve readability and future modifications.

Task 2 – Extracting Reusable Functions

Separating calculations into functions improves modularity.

Reusable functions allow the same logic to be applied to multiple inputs.

Docstrings make the code easier to understand and maintain.

Task 3 – OOP Grade Calculator

Using a class eliminates repeated conditional logic.

Encapsulation improves structure and reusability.

The method can be reused for multiple students without rewriting logic.

Task 4 – Procedural to Functions

Separating input, processing, and output improves clarity.

Functions increase reusability and testability.

Modular design enhances maintainability without changing behavior.

Task 5 – Procedural to OOP

Class-based design follows object-oriented principles.

Encapsulation protects data and organizes logic properly.
Reusable methods improve scalability for multiple employees.

Task 6 – Optimizing Search Logic

Replacing list search with set/dictionary improves time complexity.
Lookup changes from $O(n)$ to $O(1)$ average time.
Improves performance for large datasets.

Task 7 – Library Management Refactoring

Modular functions remove repeated conditional blocks.
Documentation improves clarity and professional structure.
Separating logic from execution improves maintainability.

Task 8 – Fibonacci Generator

Reusable function removes global variables.
Improves modularity and allows easy testing.
Structured logic enhances readability and efficiency.

Task 9 – Twin Primes Checker

Separating prime logic improves reusability.
Optimized prime checking reduces time complexity.
Clean modular design enhances clarity and scalability.

Task 10 – Chinese Zodiac Refactoring

Replacing if-elif with list/dictionary simplifies logic.
Separating input and processing improves modularity.
Code becomes cleaner and easier to maintain.

Task 11 – Harshad Number Checker

Returning boolean values improves reusability.
Edge case handling increases robustness.
Separation of logic and input enhances clarity.

Task 12 – Factorial Trailing Zeros

Avoiding full factorial improves efficiency.
Counting multiples of 5 reduces time complexity.
Modular design improves readability and maintainability.

Task 13 – Collatz Test Case Design

Test cases validate correctness for normal, edge, and invalid inputs.
Using pytest ensures automated verification.
Improves reliability and robustness of implementation.

Task 14 – Lucas Sequence Test Cases

Edge and negative tests ensure proper validation.
Automated tests verify correctness for large inputs.
Improves reliability and modular testing structure.

Task 15 – Vowel & Consonant Counter

Testing normal and edge cases ensures accuracy.
Handles empty and special cases properly.
Pytest validation increases confidence in results.